Rendering-Layer Integrity Verification for Adversarially Robust Resume Screening

Devansh Pant, Dhruti Dutia, Yashvi Lohiya, Vedant Agrawal
K. J. Somaiya School of Engineering, Mumbai, India

***Abstract*—Automated resume screening systems read the text layer of a candidate document and treat it as a faithful record of what the document says. We show that this assumption is unsafe. An applicant can embed job-description keywords in a resume using white-on-white text or one-point fonts, producing a file that looks unchanged to a human reviewer while injecting arbitrary content into every text extractor we tested. Across 72 attacked documents, an undefended semantic pipeline placed an attacker at rank one for every role we evaluated. We propose rendering-layer integrity verification, which inspects per-glyph fill colour, font size and PDF text rendering mode rather than the flattened text, and we report 100 percent detection on both attack variants with a 2.8 percent false positive rate on 72 clean documents. Enabling the defence removes the attacker from rank one entirely. We also separately report a transfer failure and its repair. On a corpus of 2,483 resumes across 24 occupational categories, a general-purpose cross-encoder re-ranker reduced Precision@10 from 0.800 to 0.700 relative to bi-encoder retrieval alone ($p = 0.022$), which we attribute to domain mismatch between passage-ranking pretraining and long-form resume documents. Adapting the same re-ranker with weakly supervised pairs drawn from category labels reversed the effect and raised Precision@10 to 0.862, exceeding both the undapted re-ranker ($p = 0.0013$) and the lexical baseline ($p = 0.0036$).**

***Index Terms*—resume screening, adversarial robustness, document integrity, information retrieval, dense retrieval, algorithmic hiring**

I. Introduction

Large employers receive far more applications than any human team can read. The standard response is an applicant tracking system that ranks candidates automatically, and the research literature has moved steadily from lexical matching towards dense retrieval, where resumes and job descriptions are embedded in a shared vector space [1], [2]. Work in this area is usually evaluated on ranking quality: does the system place genuinely suitable candidates near the top.

That framing contains a hidden assumption. Every pipeline we are aware of begins by calling a text extractor on the uploaded file and then treats the returned string as ground truth. The extractor discards almost everything about how the document is drawn. Colour, glyph size and the PDF text rendering mode are all thrown away before the model sees anything. An applicant who understands this can write text that the extractor reads and a recruiter cannot see.

This is not a hypothetical concern. The same idea, invisible text placed to manipulate an automated reader, has a long history in web search, where it is catalogued as a standard spam technique [3]. Resume screening has the same structure, a ranking system standing between an applicant and a scarce resource, with the same incentive to game it, but without the adversarial defences that search engines developed in response.

This paper makes three contributions.

First, we quantify the vulnerability. We construct attacked resumes by injecting role-specific keywords into off-topic documents using two rendering tricks, and we measure how far an attacker can climb an otherwise reasonable semantic ranking pipeline.

Second, we propose and evaluate a defence that operates on the rendering layer rather than the text layer, recovering per-glyph attributes that the ordinary extraction path discards, and we report its detection rate and its false positive cost on clean documents.

Third, we report a transfer failure and its repair. Adding an off-the-shelf cross-encoder re-ranker, a step close to standard practice in neural retrieval [4], measurably degraded ranking quality on our corpus. Adapting that re-ranker to the domain with weak supervision reversed the loss and produced the best system we measured.

II. Related Work

Dense retrieval. Sentence-BERT showed that siamese transformer encoders produce sentence embeddings suitable for similarity search [1], and distilled variants such as MiniLM made this cheap enough for commodity hardware [5]. Approximate nearest neighbour libraries, FAISS in particular [6], make the retrieval step practical at corpus scale. The common deployment pattern is two stages: a fast bi-encoder retrieves a candidate set, then a slower cross-encoder re-reads each query and document jointly to produce the final order [4].

Transfer of ranking models. The BEIR benchmark demonstrated that retrieval models trained on MS MARCO transfer unevenly to other domains, and that re-rankers in particular can underperform outside their training distribution [7]. Our negative result in Section V is consistent with that finding and extends it to long-form resume documents.

Fairness in algorithmic hiring. A substantial literature examines bias in automated hiring [8], [9], prompted in part by widely reported industrial failures. This work is concerned with whether a system treats candidates equitably. Our concern is adjacent but distinct: whether the input the system reads is the document the candidate actually presents.

Adversarial manipulation of ranked retrieval. Web spam taxonomies describe invisible text and keyword stuffing as established techniques for manipulating search rankings [3]. To our knowledge this threat model has not been evaluated against resume screening systems, and no published screening pipeline inspects document rendering attributes as a defence.

III. System Description

Fig. 1 shows the pipeline. The upper path is a conventional screening system and provides the setting in which we evaluate the attack. The shaded component is the contribution of this paper.

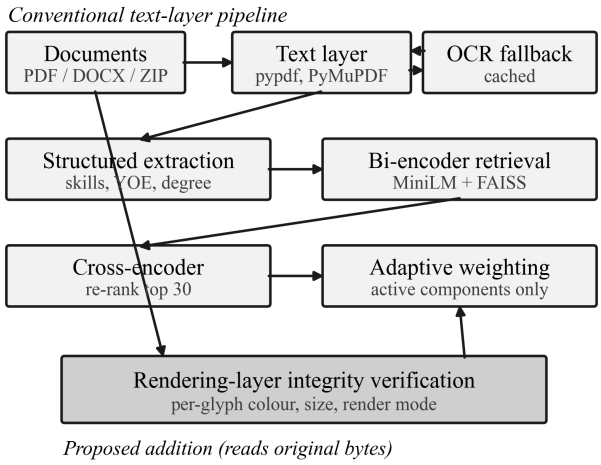


Fig. 1. Screening pipeline. The upper row is the conventional text-layer path. Rendering-layer integrity verification (shaded) reads the original document bytes rather than the extracted string, so it observes attributes the text layer discards.

A. Document ingestion

Documents arrive as PDF, DOCX or a ZIP archive. Text extraction attempts pypdf first and falls back to PyMuPDF, which recovers text from some files where the first extractor returns nothing. Scanned documents with no text layer are passed to an OCR stage. Because optical recognition costs roughly fifteen seconds per page on the hardware we target, recognised text is cached in a content-addressed store keyed by the SHA-256 digest of the file, which reduces repeat cost for an unchanged document from seconds to a few milliseconds.

B. Retrieval and scoring

Resumes and the job description are encoded with all-MiniLM-L6-v2 into 384-dimensional unit vectors and compared by inner product using a FAISS flat index, so that the inner product is exactly the cosine similarity. A cross-encoder re-ranks the top thirty candidates. Section V reports that this stage was harmful on our data.

The final score is a weighted sum of four components: semantic similarity, overlap between the skills named in the job description and those found in the resume, a comparison of years of experience against any stated requirement, and a comparison of highest degree against any stated requirement.

C. Constraint-aware weighting

A fixed weighting creates a failure mode that we encountered in practice. If a job description states no degree requirement, every candidate receives an identical education sub-score. The same holds for experience when no number of years is given. These constant terms carry no information yet occupy a fixed share of the total, which compresses the spread of final scores and, in the limit where a job description constrains none of the structured fields, makes the ranking degenerate: different job descriptions return the same ordering.

We therefore treat a component as active only when the job description actually constrains it, and renormalise the remaining weights to sum to one. Writing A for the set of active components, the score is

$$s = \Sigma_{i \in A} \left(w_i / \Sigma_{j \in A} w_j \right) \cdot c_i$$

where c_i is the ith component score. A criterion the employer never specified contributes nothing rather than contributing a constant.

IV. Rendering-Layer Integrity Verification

A. Threat model

The attacker is an applicant who controls the file they submit and wants an off-topic resume ranked highly for a target role. The attacker cannot modify the screening system and does not need to know its internals. The constraint is that the document must still look like an ordinary resume when a recruiter opens it, because a visibly keyword-stuffed document would be rejected on sight.

B. Detectors

Rather than reading the extracted string, we re-open the document and inspect each glyph. For PDF we use pdfplumber, which exposes the fill colour, size and rendering mode of every character; for DOCX we inspect run-level font properties. Four conditions raise a flag.

Near-background colour. We convert each glyph fill to perceived brightness using $Y = 0.299R + 0.587G + 0.114B$ and flag glyphs with $Y \geq 0.92$, the regime in which text is effectively invisible against white stock.

Invisible rendering mode. The PDF specification defines text rendering mode 3, which draws no marks. It has legitimate uses, notably OCR text layers placed beneath a scanned image, so on its own it is suggestive rather than conclusive.

Micro-fonts. Glyphs below three points are unreadable in print and on screen at normal zoom but extract normally.

Keyword density. A term occurring at least fifteen times and accounting for more than four percent of tokens is anomalous for a genuine resume. This detector is independent of rendering and so catches stuffing that is merely visible rather than hidden.

A flagged document is not discarded. It receives a score penalty proportional to the fraction of the document that is hidden, and the reviewer is shown the specific reason together with the extracted hidden text. We prefer this to silent rejection because the detectors can misfire, and because the recovered text is itself evidence a recruiter may wish to act on.

V. Experimental Setup

A. Corpus. We use a publicly available resume corpus of 2,483 documents labelled with 24 occupational categories, with a mean length of 6,284 characters. Applying our structured extractor, 50.7 percent of documents report a master's degree as the highest qualification, 31.7 percent a bachelor's degree and 4.1 percent none that we could identify. Explicit years of experience were absent from 61.5 percent of documents, which is the reason the experience component is disabled by default when a job

description does not state a requirement. Fig. 2 summarises both distributions.

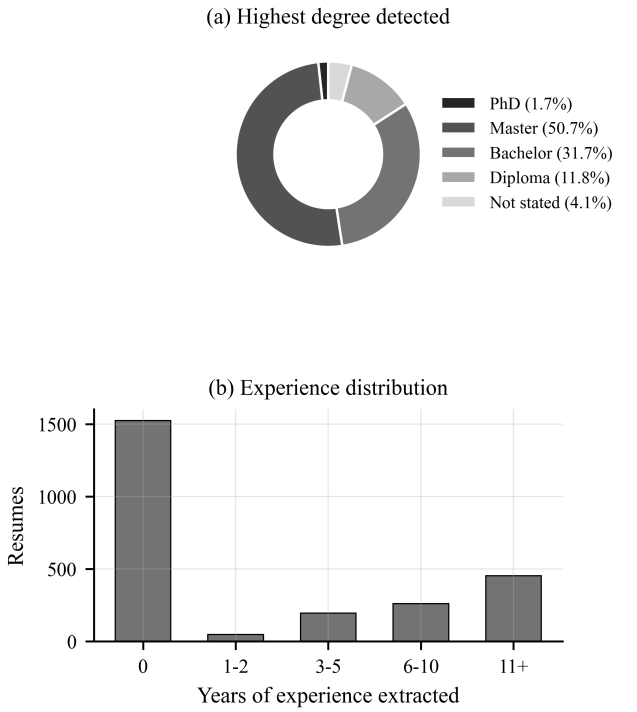


Fig. 2. Corpus characterisation. (a) Highest degree detected. (b) Years of experience recovered by the structured extractor.

B. Retrieval protocol. For each of the 24 categories we build a pool of 100 documents, 20 drawn from the target category and 80 from other categories, and issue a short job description for that role. A document counts as relevant if its category label matches the queried role. This is a coarse proxy for human relevance judgements, and we return to that limitation in Section VII; it is applied identically to every system, so the comparison between systems remains meaningful. We report Precision@10, Recall@10, NDCG@10 and MRR, and compare systems with a paired two-tailed t-test over the 24 queries.

C. Adversarial protocol. We evaluate three roles. For each, the pool contains 12 genuine resumes from the target category, 12 genuine resumes from other categories, and attacked copies of those same off-topic resumes, 12 using white-on-white injection and 12 using one-point fonts, giving 48 documents per role and 144 in total. Using the same base documents for the clean and attacked conditions isolates the effect of the injection. We score every pool twice, once with the defence disabled and once enabled.

VI. Results

A. The attack defeats an undefended pipeline

With no defence, keyword injection is decisive. An attacked off-topic resume took rank one in 100 percent of roles and occupied 100 percent of the top five positions. Attacked documents reached a mean rank of 12.5 while genuine relevant resumes averaged 37.7. The semantic pipeline is not merely degraded; it is inverted, ranking irrelevant candidates above suitable ones because the injected text is, as far as the model can tell, part of the document.

B. Detection accuracy

TABLE I. Detection rates by document class

Document class	n	Flagged	Interpretation
White-text attack	36	100.0%	true positive
Micro-font attack	36	100.0%	true positive
Clean, relevant	36	0.0%	false positive
Clean, off-topic	36	2.8%	false positive

Table I reports detection. Both attack variants were caught in every instance. One clean off-topic document of 36 was flagged, by the keyword density detector rather than either rendering detector, giving a false positive rate of 2.8 percent on that class and zero on genuine relevant documents.

C. Effect of the defence on ranking

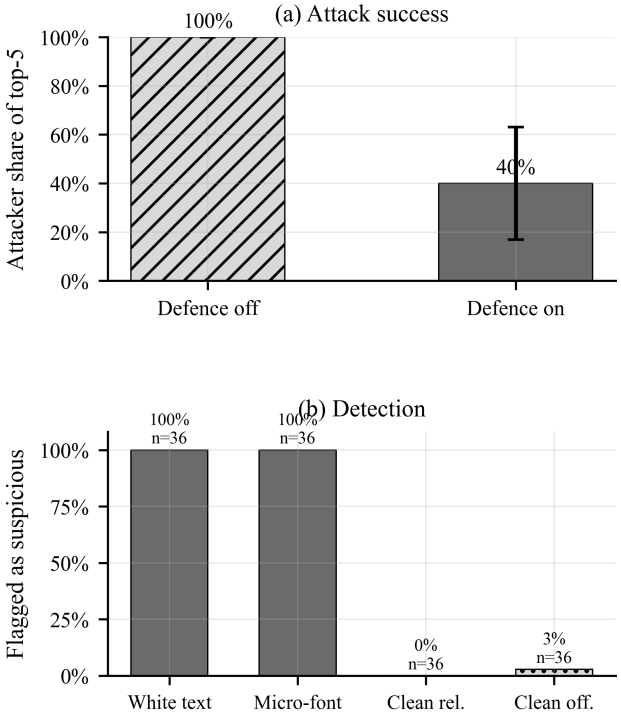


Fig. 3. (a) Share of the top five positions held by attacked documents, averaged over three roles, with and without the defence. Error bars are the standard error over roles. (b) Proportion of each document class flagged.

Enabling the defence removes the attacker from rank one in every role, from 100 percent to 0 percent. The improvement is real but partial. Attacked documents still occupy 40 percent of the top five, and their mean rank (23.9) remains marginally ahead of genuine relevant resumes (26.1). The penalty demotes an attacked document rather than eliminating it, and a resume that is off-topic but otherwise well written retains some semantic similarity to the role. We regard rank one as the operationally important case, since it is the position a shortlist is anchored on, but we do not claim the attack is fully neutralised.

D. Transfer failure and domain adaptation

TABLE II. RETRIEVAL QUALITY OVER 24 QUERIES

System	P@10	R@10	NDCG@10	MRR	s/query
TF-IDF	0.746	0.373	0.772	0.931	0.19
Bi-encoder	0.800	0.400	0.821	0.923	3.75
+ CE, pretrained	0.700	0.350	0.737	0.872	8.87
+ CE, fine-tuned	0.862	0.431	0.877	0.951	7.06

Table II gives the retrieval comparison. Bi-encoder retrieval improved on the lexical baseline, 0.800 against 0.746 Precision@10, though the difference did not reach significance over 24 queries ($p = 0.114$).

The re-ranker behaved in two opposite ways depending on whether it had seen the domain. Used as published, it *reduced* Precision@10 from 0.800 to 0.700, a mean paired difference of -0.100 ($t = -2.45$, $p = 0.022$), losing on 11 of 24 queries and winning on only 4. A practitioner following the standard two-stage recipe would have paid roughly twice the compute for a worse ranking.

We attribute this to domain mismatch. The published checkpoint is trained on MS MARCO, where both query and passage are short. Our inputs are resumes averaging 6,284 characters, truncated to the model's 512-token window, matched against paragraph-length job descriptions. Neither side of the pair resembles the training distribution, and the BEIR results [7] indicate this is a general rather than an incidental problem.

Adapting the same architecture to the domain reversed the result. Training on 1,600 weakly supervised pairs for a single epoch, with positives drawn from matching category labels and negatives from mismatched ones, raised Precision@10 to 0.862. The gain over the unadapted checkpoint is +0.163 ($t = 3.65$, $p = 0.0013$), and the adapted system also beats the bi-encoder alone (+0.063, $p = 0.016$) and the lexical baseline (+0.117, $t = 3.25$, $p = 0.0036$; 13 wins, 7 ties, 4 losses). It is also marginally faster than the unadapted model, 7.1 against 8.9 seconds per query.

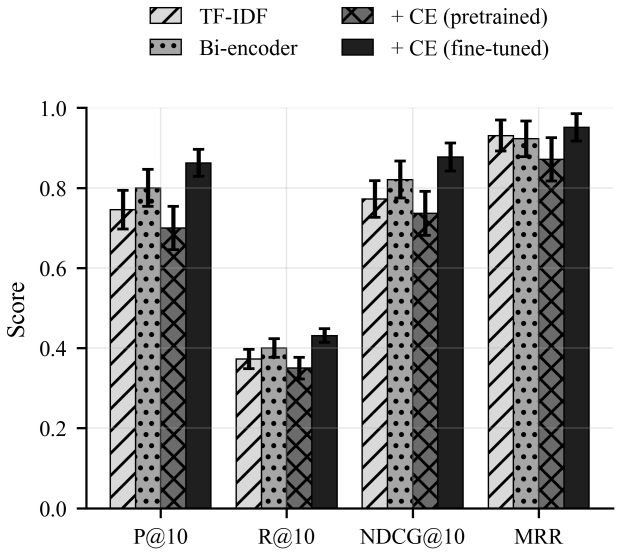


Fig. 4. Retrieval quality across the four systems. Error bars are the standard error over 24 queries.

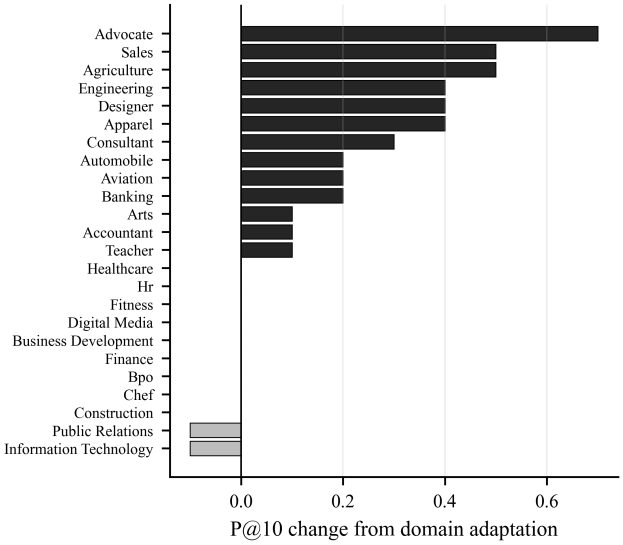


Fig. 5. Per-query change in Precision@10 from domain adaptation, sorted. Adaptation improved 13 queries and hurt 2.

Fig. 5 shows that the improvement is broad rather than driven by a few queries. We draw two conclusions. A general-purpose re-ranker should not be assumed to help, and the cost of checking is one evaluation run. When it does not help, modest domain adaptation can be enough to recover the expected benefit: a single epoch over weak labels that required no manual annotation moved this component from actively harmful to the best configuration we measured.

VII. LIMITATIONS

Our relevance labels are category matches, not human judgements. A resume from the target category may be a poor fit for a specific role and a resume from an adjacent category may be a good one, so absolute metric values should be read as approximate. The comparison between systems is unaffected because all systems see identical labels.

The brightness threshold is a fixed value. An attacker who uses a very light grey rather than pure white can fall below it, and we have not characterised the trade-off curve between threshold and false positive rate. The keyword density detector provides partial coverage in that case because it is independent of colour, but we expect a determined attacker to evade a static threshold.

Our adversarial evaluation covers three roles and two attack variants, with 144 documents. The detection rates are encouraging but the sample is small, and we generate attacked documents programmatically, so they may be more uniform than real adversarial submissions.

We did not evaluate whether the penalty magnitude is well calibrated. It was chosen so that a heavily stuffed document ranks below a clean one, which the results in Section VI-C show is only partially achieved.

VIII. CONCLUSION AND FUTURE WORK

Resume screening systems inherit a vulnerability from the way they read documents. Because text extraction discards rendering attributes, an applicant can inject keywords that the model reads and a recruiter cannot see, and in our experiments this was

sufficient to take rank one in every role tested. Inspecting the rendering layer rather than the text layer detected both attack variants in all 72 attacked documents at a false positive cost of 2.8 percent on clean off-topic documents, and removed the attacker from rank one. The defence is incomplete: attacked documents retain 40 percent of top-five positions.

We also found that a general-purpose cross-encoder re-ranker reduced ranking quality on this corpus while costing substantially more compute, which we read as a domain transfer failure rather than a flaw in the two-stage approach itself.

Three directions follow. An adaptive brightness threshold estimated from the document background would address the static threshold weakness, and an attacker who uses near-white rather than pure white is the obvious next evaluation. Our adaptation used a single epoch over weak labels, so stronger supervision, whether from recruiter feedback or from harder negative mining, is likely to improve the re-ranker further. Finally, a study with human relevance judgements would establish how far the category proxy distorts the absolute numbers reported here.

References

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982-3992.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171-4186.

[3] Z. Gyongyi and H. Garcia-Molina, "Web spam taxonomy," in *Proc. 1st Int. Workshop on Adversarial Information Retrieval on the Web (AIRWeb)*, 2005.

[4] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv:1901.04085*, 2019.

[5] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. NeurIPS*, 2020.

[6] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.

[7] N. Thakur, N. Reimers, A. Ruckle, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

[8] M. Bogen and A. Rieke, "Help wanted: An examination of hiring algorithms, equity, and bias," Upturn, Tech. Rep., 2018.

[9] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. ACM FAccT*, 2020, pp. 469-481.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333-389, 2009.